2-3 TREE INSERTION

```
#include <iostream>

using namespace std;

class Node {

public:

	int data[2];		// max 2 keys

	Node* child[3];	// max 3 children

	int n;			// number of keys

	Node(int val) {

		data[0] = val;

		n = 1;

		child[0] = child[1] = child[2] = NULL;

	}

};

class TwoThreeTree {

public:

	Node* root;

	TwoThreeTree() {

		root = NULL;

	}

	// Insert helper

	Node* insert(Node* node, int val, int &upKey) {

		if (node == NULL) {

			upKey = val;

			return NULL;
```

```
}
// Leaf node
if (node->child[0] == NULL) {
    if (node->n == 1) {
        // Insert into node
        if (val < node->data[0]) {
            node->data[1] = node->data[0];
            node->data[0] = val;
        } else {
            node->data[1] = val;
        }
        node->n = 2;
        return NULL;
    } else {
        // Split node
        int temp[3] = {node->data[0], node->data[1], val};
        // Sort
        for (int i = 0; i < 3; i++)
            for (int j = i + 1; j < 3; j++)
                if (temp[i] > temp[j])
                    swap(temp[i], temp[j]);
        node->data[0] = temp[0];
        node->n = 1;
        Node* newNode = new Node(temp[2]);
        upKey = temp[1];
```

```
            return newNode;
        }
}
// Internal node
int pos;
if (val < node->data[0])
        pos = 0;
else if (node->n == 1 || val < node->data[1])
        pos = 1;
else
        pos = 2;
int newKey;
Node* newChild = insert(node->child[pos], val, newKey);
if (newChild == NULL)
        return NULL;
// Insert into internal node
if (node->n == 1) {
        if (pos == 0) {
                node->data[1] = node->data[0];
                node->data[0] = newKey;
                node->child[2] = node->child[1];
                node->child[1] = newChild;
        } else {
                node->data[1] = newKey;
```

```
            node->child[2] = newChild;
        }
        node->n = 2;
        return NULL;
    } else {
        // Split internal node
        int tempKey[3] = {node->data[0], node->data[1], newKey};
        Node* tempChild[4] = {
            node->child[0],
            node->child[1],
            node->child[2],
            newChild
        };
        // Sort keys
        for (int i = 0; i < 3; i++)
            for (int j = i + 1; j < 3; j++)
                if (tempKey[i] > tempKey[j])
                    swap(tempKey[i], tempKey[j]);
        node->data[0] = tempKey[0];
        node->n = 1;
        Node* newNode = new Node(tempKey[2]);
        newNode->child[0] = tempChild[2];
        newNode->child[1] = tempChild[3];
        upKey = tempKey[1];
        return newNode;
```

```
        }
    }
    void insert(int val) {
        int upKey;
        Node* newChild = insert(root, val, upKey);
        if (newChild != NULL) {
            Node* newRoot = new Node(upKey);
            newRoot->child[0] = root;
            newRoot->child[1] = newChild;
            root = newRoot;
        }
    }
    // Inorder traversal
    void inorder(Node* node) {
        if (node == NULL) return;
        if (node->n == 1) {
            inorder(node->child[0]);
            cout << node->data[0] << " ";
            inorder(node->child[1]);
        }
else
{
            inorder(node->child[0]);
            cout << node->data[0] << " ";
            inorder(node->child[1]);
```

```
                cout << node->data[1] << " ";
                inorder(node->child[2]);
            }
        }
        void display() {
            inorder(root);
            cout << endl;
        }
};
int main() {
        TwoThreeTree t;
        int arr[] = {10, 20, 5, 6, 12, 30, 7, 17};
        for (int x : arr)
            t.insert(x);
        cout << "Inorder traversal of 2-3 Tree:\n";
        t.display();
  return 0;
}
```

Output

```
Inorder traversal of 2-3 Tree:



=== Code Execution Successful ===
```